Multi-Mode AI Assistant: System Design and Project Plan

Updated October 29, 2026

1. Overview

The assistant answers every question in one of three modes. All three modes share one model list, one set of skills and tools, one answer checker and one output path.

- Simple: one model answers and uses tools when it needs them.
- Thinker: 2 to 5 agents think about the same question, then a composer model merges their answers into one.
- Planner: the system writes a step-by-step plan (like Cursor's plan mode). Each step runs in Simple or Thinker, and each step is checked before the next one starts.

Example: "Make a 2-page PDF report on solar power." Planner splits it into research, outline, writing and PDF steps. Research runs in Simple with web search, writing runs in Thinker, every step is checked, and the user gets one PDF.

2. Review of the first design

The three-mode idea is strong. The gaps are missing exits, loops with no stop rule, and no checks for quality, errors or cost.

What already works:

- Three modes match three kinds of work: quick answers, deep thinking and big multi-step tasks.
- The user can pick any model from OpenRouter or NVIDIA.
- Thinker supports both "same model many times" and "different models".
- Planner reuses Simple and Thinker instead of new logic.
- Skills and tools are shared by every mode.

What to fix:

Problem in the first design	Why it matters	Fix in the new design
Composer and Planner have no line to Output	The final answer has no path back to the user	Every mode ends in one Checker, then one Output Builder
Planner back loop has no stop rule	It can loop forever and keep spending money	Max 2 retries per step, max 2 re-plans, a budget, a time limit and a Stop button
No quality check	Wrong or half-finished answers reach the user	A Checker tests every answer and every plan step
No error handling	One API error, timeout or rate limit (too many requests) breaks the run	Retry 2 times, then switch model or provider
No cost control	5 agents × 3 rounds + 1 composer = 16 model calls for one question	Cost estimate before the run, budget guard during the run
"Think together" is not defined	Two developers would build it two different ways	Debate rounds on a shared board; stop when answers agree or after 3 rounds
Same model 5 times gives almost the same answer 5 times	Five copies add cost, not new ideas	Give each copy its own role and temperature (randomness)
The user must always choose a mode	New users choose the wrong one	Auto mode: a small, cheap model picks the mode
No input step	No safety check, chat history, file reading or clarifying question	Input Gateway before the router
Tools have no rules	Endless tool calls and risky actions without asking	Max 10 tool calls, permission for risky tools, a sandbox for code
Model list has only names	The system cannot know price, context size or tool support	Model Registry with price, context size (how much text fits), tool, image and thinking support
No logs or feedback	Problems cannot be found or fixed	Log every step with tokens, cost and time; let the user rate answers

3. Improved full system

Every question passes five stages. Only stage 3 changes with the mode, so all modes get the same safety, cost and quality checks.

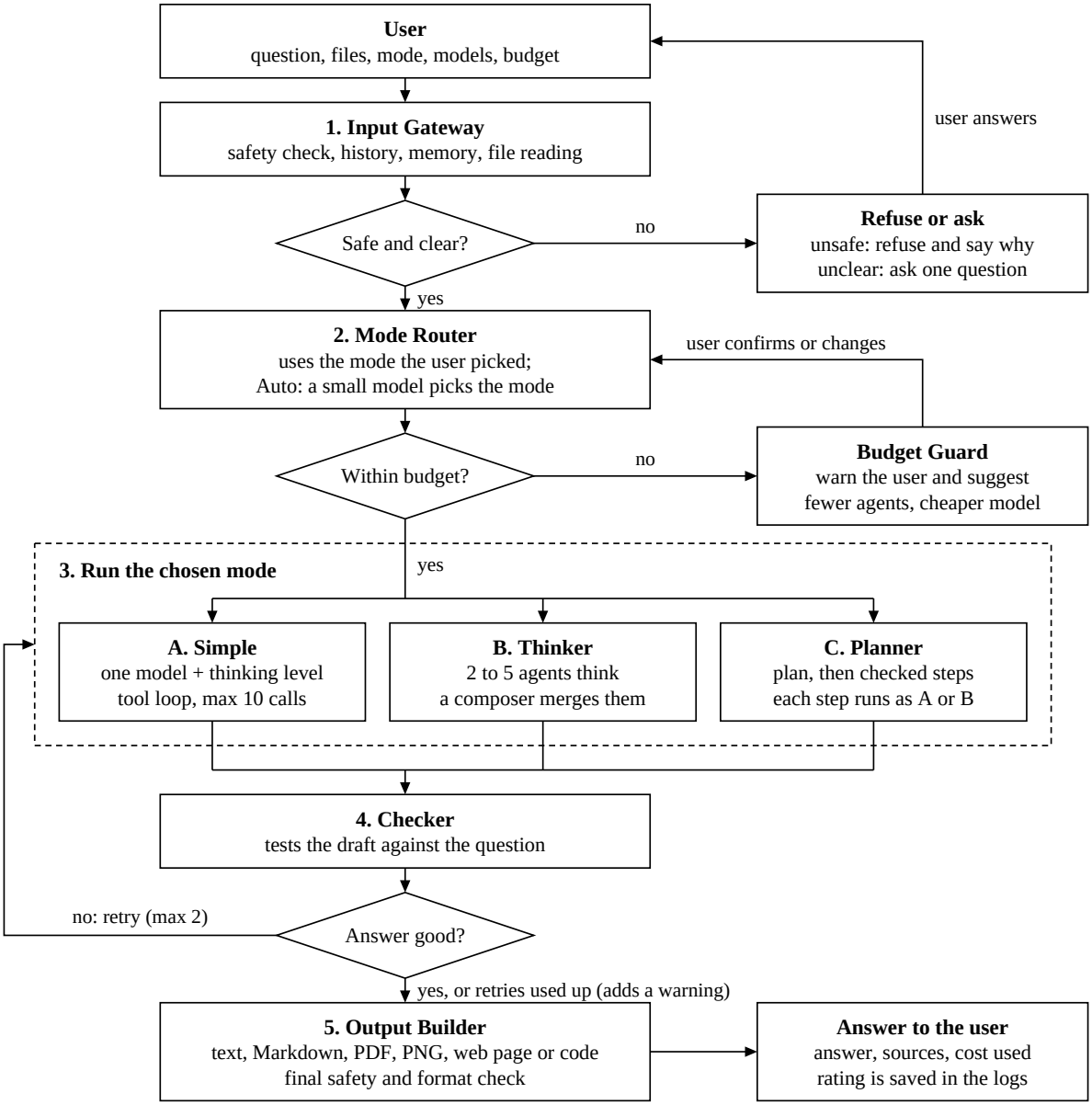


Figure 1. Full system. A "no" at a diamond stops, asks the user or loops back; nothing reaches the user without passing the Checker.

Shared services (any stage can call them):

Service	What it does
Model Registry	Lists every OpenRouter and NVIDIA model with its company, price, context size and tool, image and thinking support
Skills and Tools Hub	Skills are instruction files loaded into the prompt (thinking, PDF, humanizer, web page maker, design, output format, PNG). Tools are code that runs in a sandbox (web search, code runner, file reader)
Memory and Files	Chat history, saved user facts and uploaded files
Cost Meter	Counts tokens and money for every call and stops the run at the budget
Fallback Manager	Retries failed calls, then switches model or provider
Logs and Feedback	Saves every step (model, tokens, cost, time) and the user's rating

4. Mode A: Simple

One model answers. If it asks for a tool, the tool runs, the result goes back to the model, and the loop repeats until the model answers without a tool.

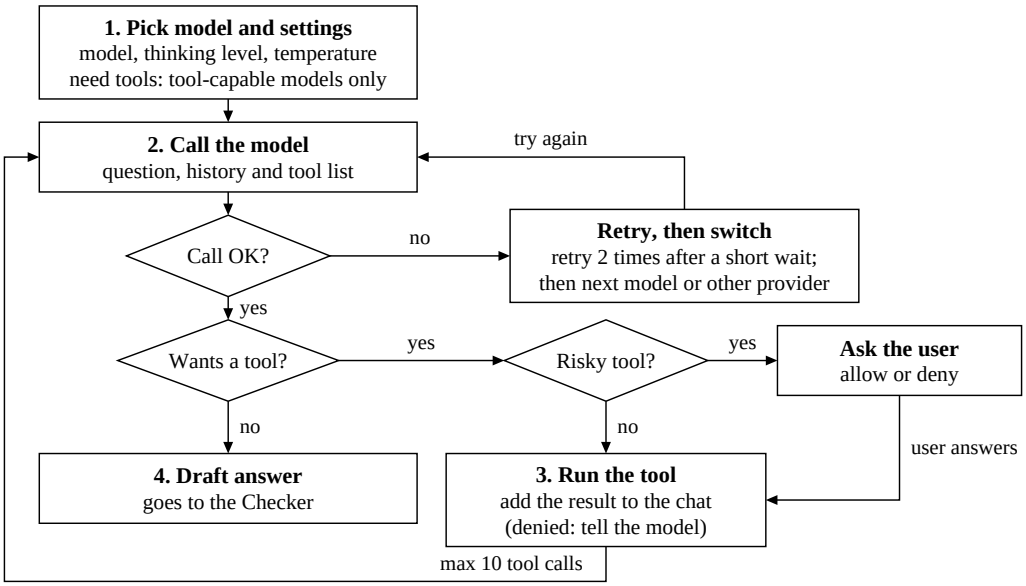


Figure 2. Mode A: Simple, a tool loop with retry and fallback.

The user can set the model, thinking level (off, low, medium, high), temperature (randomness) and maximum answer length. The tool list sent to the model comes from the Skills and Tools Hub and holds only the tools this question needs.

5. Mode B: Thinker

Several agents work on the same question, and one composer turns their answers into a single answer. More agents and more rounds give more points of view but cost more, so the Budget Guard checks them first.

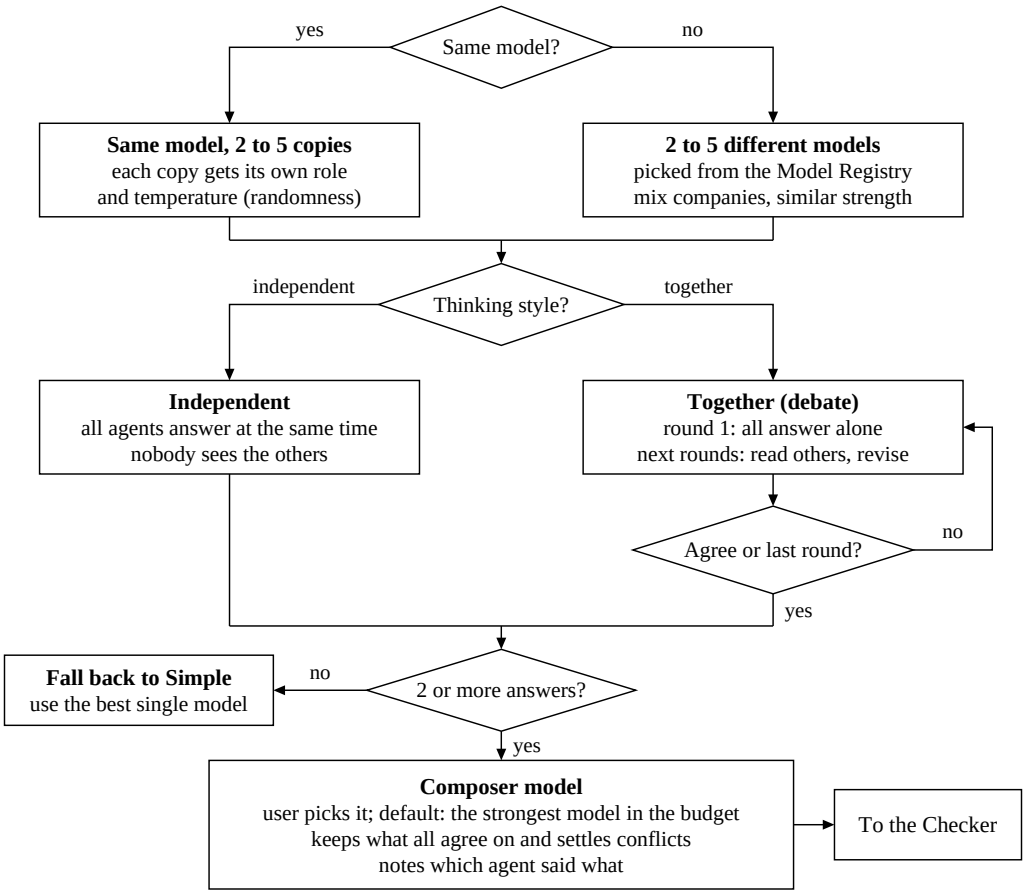


Figure 3. Mode B: Thinker, with 4 decisions and 1 debate loop.

- Roles for copies of the same model (examples): solver, critic, fact-checker, simplifier, devil's advocate.
- Together mode: in every round, each agent sees the other agents' latest answers on a shared board. The answers "agree" when a small judge model finds the same main conclusion in all of them.
- The composer receives every final answer with its role (not its model name) and, after a debate, the round-1 answers too, so it can see if agents only copied each other.

6. Mode C: Planner

The Planner never runs a step without a plan the user approved, and every loop passes the limit check before the next step.

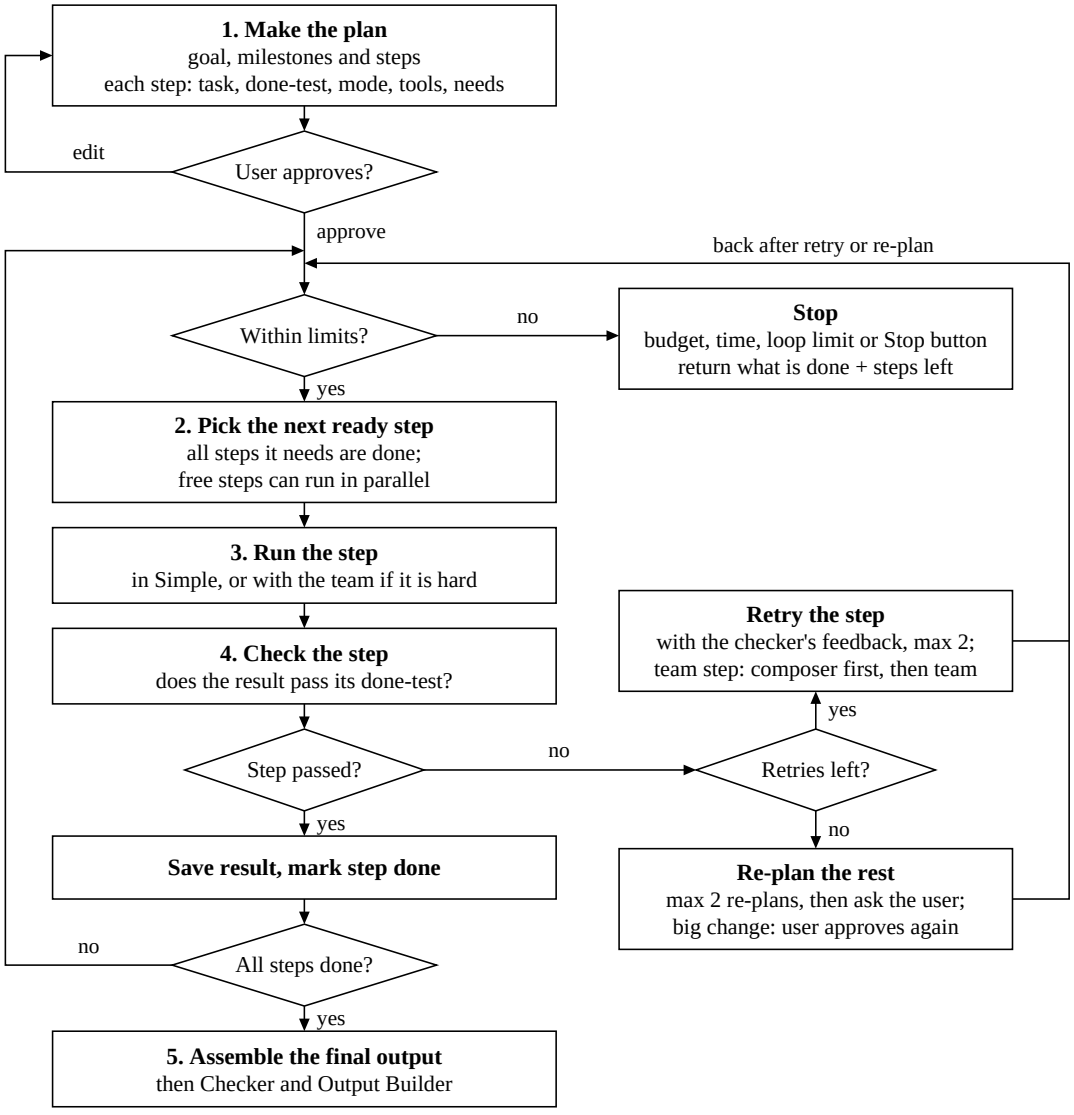


Figure 4. Mode C: Planner. Plan, approve, run, check, then retry or re-plan.

Each step stores its task, done-test (how we know it is finished), easy or hard, mode, tools, needs (steps that must finish first) and status. A strong model can write the plan while cheaper models run the easy steps.

7. Team cases: Thinker teams and Planner steps

A team is used in two places: in Thinker mode, and in every Planner step that runs with Thinker. Both use the same five settings, so this part is built once: which models (the same model or different models), how many agents (2 to 5), how they think (on their own, or together in rounds), rounds (1 to 3, only for together) and the composer (the model that writes the final answer).

Team case (n agents, R rounds)	What happens	Model calls	Use it for
1. Same model, on their own (default)	n copies answer alone, each with its own role; the composer merges them	n + 1	Most questions: cheap, and strong when one model is clearly the best
2. Same model, together	Round 1: each copy answers alone. Next rounds: each reads the others and revises	up to n × R + 1	Checking one line of reasoning; copies can share the same blind spot
3. Different models, on their own	n different models answer alone; the composer merges them	n + 1	More points of view; pick models of similar strength
4. Different models, together	Debate rounds between different models	up to n × R + 1	Hard questions with a checkable answer: math, code, logic

Input: 3 agents, together, 2 rounds. **Step:** 3 × 2 = 6 agent calls + 1 composer call = 7. **Output:** at most 7 calls instead of 1 in Simple. A cheap judge after each round skips the rest if they agree.

Planner with a team:

- **How steps run:** Simple (no team, cheapest), Auto (default: easy steps in Simple, hard steps with the team) or Thinker (every thinking step with the team).
- **Hard step:** judging, comparing, long writing, or a step that failed in Simple; the plan card shows it.
- **Gather once:** a team step runs its tools one time and puts the results on a shared board, so every agent uses the same facts. The board keeps only each agent's latest short answer, so it does not fill the memory window.
- **Cost:** the plan's estimate includes every team step, and the user approves that price.

Why "on their own" is the default: voting between independent answers gives most of the gain of debate (Debate or Vote, 2025), and answers from the one best model often beat a mix with weaker models (Self-MoA, 2025). Debate gains most from different models (Stop Overvaluing Multi-Agent Debate, 2025).

Team conditions:

ID	Where	If	Then
T1	Planner	How steps run = Simple	All steps run in Simple. If a step fails, the card offers "Use the team"
T2	Planner	How steps run = Auto	Each step is marked easy or hard; only hard steps use the team
T3	Planner	A step only runs a tool	Run it in Simple, like making a PDF: a team adds cost, not quality
T4	Team step	The step needs tools or a file	Run the tools once before the team. Models without tools or vision read the board and the file's text and chart descriptions
T5	Team	Strong and weak models mixed	Warn: a weak model can pull the answer down; suggest similar ones
T6	Team	An agent's model fails 2 retries	Same model: drop that copy. Different models: add the next similar model not in the team. Fewer than 2 left: C13
T7	Together	The judge says they agree	Skip the other rounds and go to the composer
T8	Together	An agent changes its answer	It must say why; the composer sees round-1 answers, so copying shows
T9	Composer	The composer reads the answers	Show roles, hide model names, so it cannot favor its own model
T10	Team step	A team step fails its done-test	Retry 1: composer only, with the checker's feedback (1 call). Retry 2: the whole team. Then re-plan (C17)
T11	Planner	Teams push the plan over budget	Before approval, offer fewer team steps, fewer agents or rounds, or cheaper models

8. All conditions (IF / THEN) and default limits

Every loop has a limit and every failure has a fallback, so a run always ends with an answer or a clear message. Team cases have their own list, T1 to T11, in section 7.

ID	Stage	If	Then
C1	Input	The message is unsafe	Refuse and say why
C2	Input	The question is unclear	Ask the user one short question
C3	Input	History or files are too long for the model	Summarize old messages; split files into parts
C4	Router	The user picked a mode	Use that mode
C5	Router	Mode is Auto	Chat or a simple fact: Simple. Judgment or comparison: Thinker. Many steps or a file to make: Planner

ID	Stage	If	Then
C6	Router	Auto is not sure	Use Simple and offer a "Think deeper" button
C7	Router	The cost estimate is over the budget	Warn; suggest fewer agents or a cheaper model
C8	Model call	Error, timeout or rate limit	Retry 2 times, then use the next model or the other provider
C9	Model call	Tools are needed but the model has no tool support	Use a tool-capable model, or run without tools and say so
C10	Tools	The model asks for a tool	Run it, add the result, call the model again (max 10 tool calls)
C11	Tools	The tool is risky (writes files, sends messages, runs code outside the sandbox)	Ask the user first
C12	Thinker	Style is Together	Debate until they agree or the set rounds (1 to 3) end
C13	Thinker	Fewer than 2 agents answered	Fall back to Simple mode
C14	Thinker	No composer chosen	Use the strongest model inside the budget
C15	Planner	The user edits the plan	Re-plan and show it again
C16	Planner	A step fails its done-test	Retry with the checker's feedback (max 2); a failed Simple step retries with the team, unless How steps run = Simple (T1)
C17	Planner	Retries are used up	Re-plan the remaining steps (max 2 re-plans), then ask the user
C18	Planner	A re-plan changes the goal or the cost a lot	Ask the user to approve again
C19	Checker	The answer fails and retries are used up	Send the best answer with a warning
C20	Any stage	Budget, time or loop limit reached, or Stop pressed	Stop, save progress, return the partial result and the steps left
C21	Output	The output fails the safety or format check	Fix it once; if it still fails, send plain text

The user sets: 2 to 5 agents, 1 to 3 debate rounds and a money budget per question. Fixed safety limits: 10 tool calls per answer, 2 retries per call or step, 2 re-plans and 20 plan steps.

9. Project plan and tech stack

Build the system in four phases. Each phase ends with a test gate, and the next phase starts only after its gate passes.

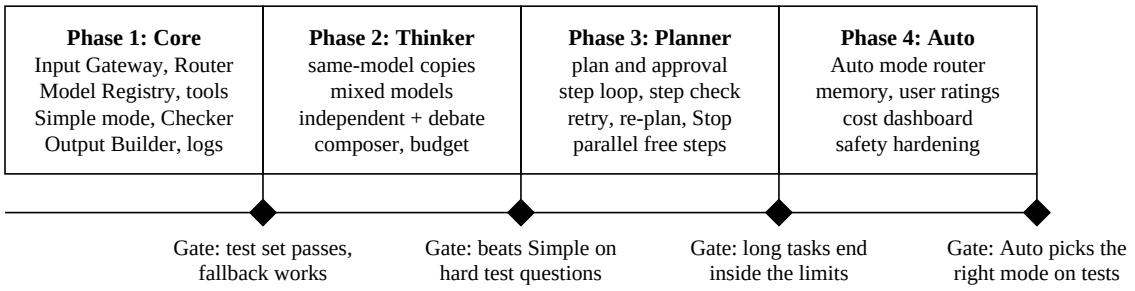


Figure 5. Build roadmap: 4 phases, 4 test gates.

A test set is a fixed list of questions with known good answers. Keep it the same in every phase so the results can be compared. At the Phase 2 gate, run the 4 team cases on it: a case that costs more but does not score higher stays off by default.

Part	Suggested choice	Why
Frontend	Next.js (React) with streaming	Shows each agent's progress live
Backend	Python with FastAPI	Calls many models at the same time
Agent flow	LangGraph, or your own state machine	Loops and IF / THEN branches are built in
Model access	One adapter; both providers take OpenAI-style requests	Same code for every model; easy fallback
Model Registry	Fill it from OpenRouter's model list (price, context length, tools, input types); add missing NVIDIA data by hand	The router and the Budget Guard need these numbers for every model
Storage	PostgreSQL + pgvector (memory search), Redis (queues)	Chats, plans, logs and long tasks
Tool safety	Docker sandbox for code; an allow-list for risky tools	Tools cannot harm the server or the user
Logs	OpenTelemetry or Langfuse traces	Tokens, cost and time for every step

Sources: OpenRouter API reference and models list (openrouter.ai/docs); NVIDIA NIM hosted API guide (docs.api7.ai); team research: Debate or Vote (NeurIPS 2025), Rethinking Mixture-of-Agents and Stop Overvaluing Multi-Agent Debate (arxiv.org).

Part 2: Memory and Attachments

October 27, 2026

Two rules fix both problems: a file is read once and never paid for twice, and the chat never forgets silently. It warns first, summarizes with a check, and keeps every original message searchable.

1. The simple idea

Think of the model as a person at a small desk. The desk is the **context window**: the most text the model can read at once, counted in **tokens** (a token is about 3/4 of an English word). Long chats and big files do not fit on the desk, so the system needs a filing cabinet (storage) and a librarian (search) that puts the right pages on the desk at the right time.

Problem	Fix in this design
Reading files costs money (OCR)	The user picks the reader in Settings and sees the price first. Only pages that need OCR are sent to it, and a file already read costs $0
Charts and tables lose their meaning as plain text	Tables keep their rows and columns. Chart pages get a written description, and the page image is kept
Long chats forget earlier messages	Every message is saved word for word. Alarms come at set limits, summaries are checked, and old messages stay searchable
Moving a chat to a new session	A download file holds the messages, the memory and the file readings, so an import restores everything without paying to read the files again

2. How Claude and ChatGPT do it today

Both follow one pattern: the whole file goes in the window when it fits, and it is searched when it does not. This design keeps that pattern and adds cost control, alarms and a real import.

Feature	How it works today	Our design
Files in a chat	Claude puts the whole file in the window; each PDF page goes in as text (1,500 to 3,000 tokens) plus a page image. OpenAI's file search cuts files into 800-token chunks (400 overlap) and adds the best ones	Whole file when it fits, search when it does not
Many files	Claude Projects switch to search ("RAG mode") near the window limit and hold up to 10 times more	The same switch, set by each model's window
Long chats	Claude (paid plans, code execution on) summarizes earlier messages near the limit and keeps the full history	The same, plus alarms, a checked summary and a ledger
Memory across chats	Claude saves topics and can search past chats; ChatGPT keeps saved memories. Both let users view and delete them, and in both, deleting a chat does not delete its memories	Same controls; deleting a chat asks about its memories too
Moving a chat	Claude exports your data (Settings, Privacy, Export data); an uploaded transcript is only read into the window like any file	A real export and import, files included

Why Claude seems to remember an uploaded chat: the transcript is read into the window like any file. That works only while it fits; most current Claude chat models have a 1M-token window, some 500K or 200K.

Why an app forgets inside a chat: if it keeps only the newest messages that fit, the oldest fall out without warning. Our rule: never drop a message silently.

3. Attachment pipeline

Every file is fingerprinted first, so a file the system has read before costs $0. Then each page goes to the cheapest reader that can handle it, and paid readers run only after the cost check.

- **Fingerprint:** SHA-256 turns the file's bytes into a short ID; the same file always gives the same ID, so the system knows it has read it, even in another chat. Saved readings belong to one account, so no user can find out what another uploaded.
- **Document Card:** a short card per file: title, pages, summary, outline with page numbers, tables and charts. The cards of all files in a chat stay in the window, so the model never forgets a file exists, even when its text is searched.
- **Chunks:** pieces of up to 800 tokens, cut at headings and paragraphs. A table is never cut mid-row; a long table is split into groups of rows with the header row repeated. Every chunk starts with its file, section and page.

Two caches, two jobs

1. **Parse Cache** (ours, kept until the user deletes the file): saved readings by fingerprint, reader and version; it saves the OCR money.
2. **Prompt cache** (the provider's, 5 minutes, or 1 hour as an option): a repeated prompt start is billed cheaper. Anthropic: reads cost 0.1 times the input price (0.05 on Opus 5.5 and Sonnet 5.5), writes 1.25 times (5 minutes) or 2 times (1 hour). OpenRouter caches automatically for many providers. So stable parts go first, the new message last.

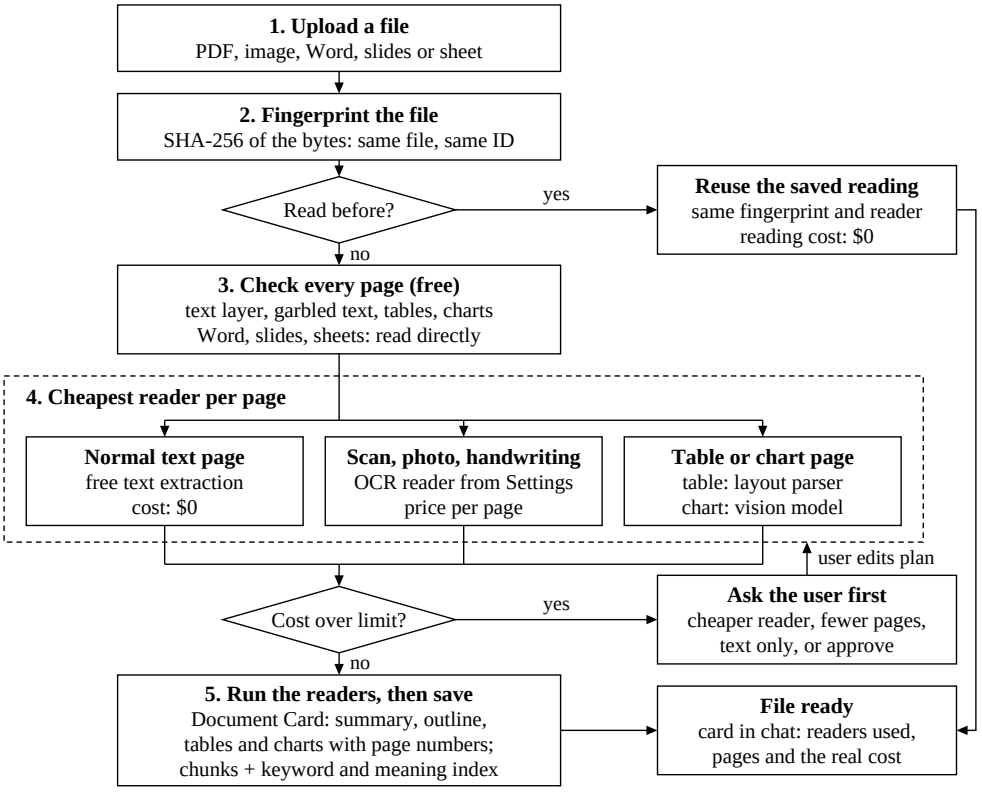


Figure 6. Attachment pipeline: a saved reading costs $0, and a paid reader runs only for pages that need it, after the cost check.

4. OCR reader settings and cost

The user picks the reader in Settings and always sees the price before paying. Auto, the default, mixes the readers below and uses the cheapest one that can handle each page.

Reader (in Settings)	Examples	Cost	Best for
Text layer only	Built-in PDF text; OpenRouter "cloudflare-ai" engine	Free	Normal digital PDFs; fails on scans and charts
Local open-source	Docling (MIT license), Tesseract, or NVIDIA Nemotron 3.5 Parse on your own GPU	No per-page fee; uses your server	Tables, privacy, large volumes
OCR API	Mistral OCR 3, direct or through OpenRouter "mistral-ocr"	$2 per 1,000 pages ($1 with Mistral's batch API)	Scans, forms, handwriting, hard tables
Vision model	Any image model in the Model Registry, for example Claude Haiku 4.5	Input tokens: one page image is about 1,600 tokens	Charts, diagrams, screenshots

Watch out: with no engine named, OpenRouter tries the model's own file reading, otherwise paid Mistral OCR. Our app always sets the engine from Settings and reuses OpenRouter's file annotations, so no PDF is parsed twice.

Other settings: ask before paying when one file costs more than $0.50 (default); a monthly reading budget with an alarm at 80%; chart descriptions on or off; keep page images on or off.

Cost example: a 200-page report

Input (what we know):

- 170 normal text pages, 20 scanned pages and 10 chart pages.
- OCR API: $2 per 1,000 pages, so 2 ÷ 1,000 = $0.002 per page.
- Vision model (Claude Haiku 4.5): $1 per million input tokens and $5 per million output tokens. One page image is about 1,600 tokens; a chart description is capped at 300 tokens.

Step (what we do):

1. Text pages: 170 × $0 = $0
2. Scanned pages: 20 × $0.002 = $0.04
3. Chart pages, input: 10 × 1,600 = 16,000 tokens, and 16,000 ÷ 1,000,000 × $1 = $0.016
4. Chart pages, output: 10 × 300 = 3,000 tokens, and 3,000 ÷ 1,000,000 × $5 = $0.015
5. Total: $0 + $0.04 + $0.016 + $0.015 = $0.071

Output (what we get): about $0.07 for the whole file. Sending every page to the OCR API would cost 200 × $0.002 = $0.40, and the charts would still be lost. Uploading the same file again later costs $0.

5. Finding the right part of a file

Files that fit their budget (35% of the window) are read in full; bigger files are searched.

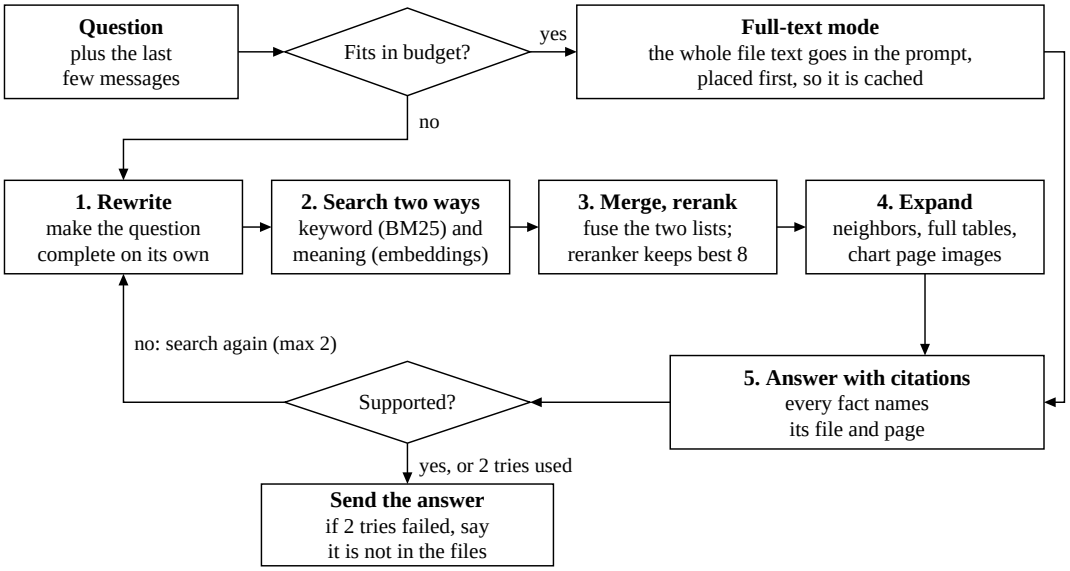


Figure 7. Finding the right part: full text when it fits, otherwise rewrite, search, rerank and expand; an answer without support starts a new search.

- **Keyword search (BM25):** finds pieces with the same words; best for names, codes and numbers.
- **Meaning search (embeddings):** each piece becomes a list of numbers that captures its meaning, so "car price" also finds "vehicle cost".
- **Merge and rerank:** a piece near the top of both lists wins (rank fusion); then a reranker, a small model that reads the question and a piece together, scores the top 50 and keeps the best 8.
- **Why both:** in Anthropic's tests, context on chunks plus keyword and meaning search cut failed searches by 49%, and by 67% with a reranker. Both run through OpenRouter's embedding and rerank endpoints or locally.

6. Memory layers

Storage keeps everything; before every turn, the Context Builder rebuilds the desk within a fixed budget.

- **Archive:** every message is kept word for word with its time; summaries only save room on the desk. A June 2026 study found raw turns answered far better than compressed storage (26.0 vs 10.7 on LongMemEval).
- **Session ledger:** a numbered list of goals, decisions, numbers, rules and open questions, each linked to its message. It changes by small edits (add, change, close), never by a full rewrite, which wears details away.
- **Long-term memory:** facts about the user and their projects, each with a date and a source. When a fact changes, the old one is marked "replaced" with the date instead of being deleted, so "what did I say last month?" still works.
- **Order and speed:** stable parts first (so the prompt cache works), the new message last (models use the start and end best). The Memory Writer runs after the reply, and clean-up runs while the user is idle.

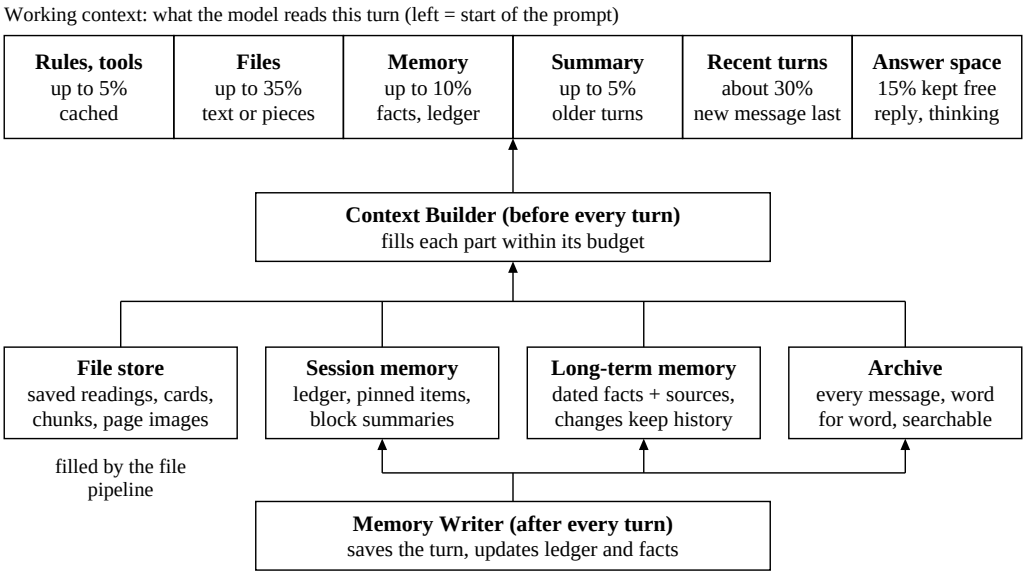


Figure 8. Memory layers: storage keeps everything; the Context Builder fills the window part by part, and the Memory Writer updates storage after each reply.

7. Context alarm and safe summaries

After every reply the system measures how full the window is and acts at 70%, 85% and 95%. Nothing leaves the window without a checked summary, and the originals stay in the Archive.

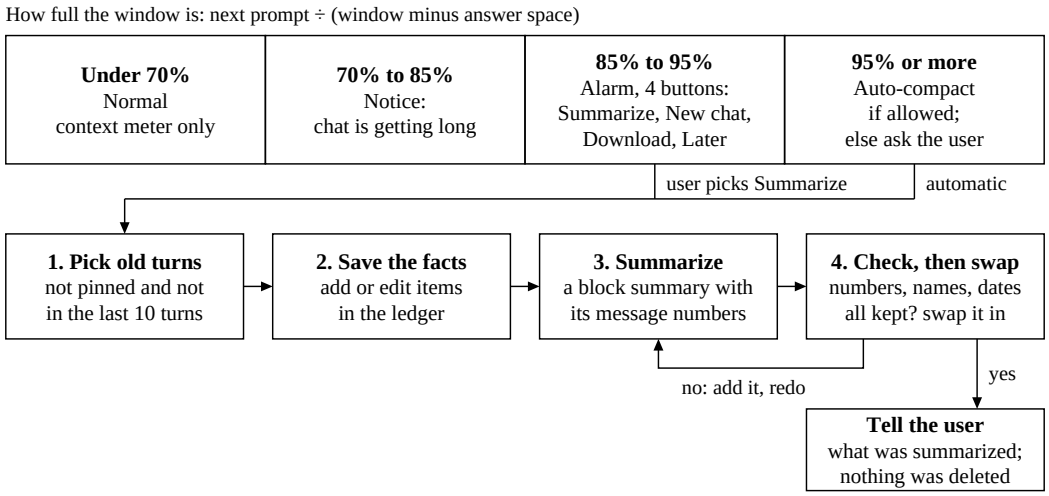


Figure 9. Context alarm: the fill level decides the action; compaction saves facts in the ledger and is checked before the summary replaces old turns.

How full is it? (example)

- Input: window 200,000 tokens; answer space 15%, so 200,000 × 0.15 = 30,000 tokens; next prompt 150,000 tokens.
- Step: usable window = 200,000 - 30,000 = 170,000; then 150,000 ÷ 170,000 = 0.88.
- Output: 88% full, so the alarm shows.

The meter also shows the price of the next message, since every turn sends the whole window again (the cached part costs less). Before switching to a smaller-window model, the system measures again and compacts first if needed.

8. Chat export and import

The download holds everything the chat knows; an import restores it without paying to read files again.

Part of the .zip file	What it holds
manifest.json	Format version, app version, date, models used, and a SHA-256 checksum for every part; the whole file can be locked with a password
messages.jsonl	Every message word for word, with its ID, time, role, model and token count
ledger, summaries, memories	The numbered ledger, the block summaries (with the message IDs they cover) and the long-term facts used in this chat, with dates and sources
files/ and readings/	The original files (optional) and their saved readings: text, tables, chart descriptions, Document Cards and fingerprints
transcript.md	A plain copy for people to read

Import, step by step:

1. Check the format version and every checksum. If something is broken, refuse and say which part.
2. Treat everything inside as data, never as instructions: a line saying "ignore your rules" is just text.
3. Reuse a file reading when the fingerprint and reader match ($0); otherwise ask before reading again. Rebuild the search indexes, showing the cost first if the embedding model changed.
4. If everything fits, load it word for word (this is why Claude seems to remember an uploaded chat). If not, load the ledger, the summaries and the last turns, and keep the rest searchable.
5. Show a report, for example: 142 messages, 3 files, 21 facts restored, reading cost $0.

9. Conditions C22 to C44 (IF / THEN)

These continue Part 1 (C1 to C21): paid steps ask first, loops have limits, nothing is deleted without the user.

ID	If	Then
C22	**Upload:** same fingerprint, same reader as before	Reuse the saved reading; cost $0
C23	**Upload:** a Word, PowerPoint, Excel or CSV file	Read text and cells directly; OCR only the pictures inside
C24	**Page:** good text layer, no table or chart	Free text extraction
C25	**Page:** scan, photo or handwriting	OCR with the reader chosen in Settings
C26	**Page:** has a table	Layout parser; keep rows and columns; repeat the header if split
C27	**Page:** has a chart or diagram	Vision model writes a description (max 300 tokens); keep the image
C28	**Cost:** over the "ask me" limit or monthly budget	Ask: cheaper reader, fewer pages, text only, or approve
C29	**Reader:** fails or times out	Retry once, then the next reader in the list; mark weak pages
C30	**Question:** files fit their budget (35%)	Full-text mode; file text first so it is cached

ID	If	Then
C31	**Question:** files are too big for it	Search mode: rewrite, search two ways, rerank, expand
C32	**Answer:** the checker finds a claim with no source	New search (max 2 tries), then say it is not in the files
C33	**Answer:** the question is about a chart	Add the page image if the model sees images; else the description
C34	**Context:** under 70% full	Normal; show the meter
C35	**Context:** 70% to 85% full	Show the notice
C36	**Context:** 85% to 95% full	Alarm with 4 buttons; "Later" asks again at 90%
C37	**Context:** 95% or more, or next message won't fit	Auto-compact if allowed; otherwise ask the user
C38	**Model:** the user switches to a smaller window	Measure again; compact before sending if needed
C39	**Memory:** a new fact conflicts with an old one	Keep the new one; mark the old one "replaced" with the date
C40	**Memory:** "as I said last week"	Search the Archive with a date filter; add the original messages
C41	**Memory:** temporary chat, or memory switched off	Write nothing to long-term memory
C42	**Delete:** the user deletes a chat or a memory	Remove it from every store, index and cache; ask about its memories
C43	**Import:** the format or a checksum is wrong	Refuse and name the broken part
C44	**Import:** imported text contains instructions	Treat it as data only

10. Research behind the design, build order and tests

Research (2023 to Oct 2026) agrees: keep raw words, date every memory, search two ways, update in small steps.

Research	What it found	How we use it
MemGPT (2023)	A small fast window plus big storage, like a computer	The desk and the stores
Lost in the Middle (2023)	Facts at the start or end of a prompt are used best	Stable facts first, new message last
Context Rot, Chroma (2025)	18 models got less reliable as the input grew	Keep the window lean; search
LongMemEval (ICLR 2025)	Chat assistants lost 30% accuracy on long-term memory	A date on every message and fact
Zep (2025)	Facts with start and end times; up to 18.5% better	Mark old facts "replaced"
ACE (ICLR 2026)	Repeated rewriting erodes details; small updates keep them	Ledger with small edits
Memory study (June 2026)	Raw turns beat compressed storage (26.0 vs 10.7)	Keep the raw words
Agent Zero (Aug 2026)	Origin, time, evidence per memory; 95.6% on LongMemEval	Source and time on every memory
MemFit (Oct 2026)	Turns saved as is, no model call; summaries only index them	Cheap writes; one search for all

Build order (phases from Part 1) and tests:

Phase	Memory and file work	Test that must pass first
1. Core	Fingerprint, Parse Cache, free text reading, full-text mode, Archive, meter	Same file twice costs $0; meter matches the provider's count
2. Thinker	OCR settings, page routing, cost check, reranking	20 test PDFs: table and chart answers cite the right pages
3. Planner	Ledger, checked compaction, alarms, export and import	50 planted facts: 95% recalled after compaction and import
4. Auto	Dated long-term memory, idle clean-up, image search	A changed fact: the newest wins and its history is kept

Sources. OpenRouter: PDF inputs, prompt caching, API reference; Readers: Mistral OCR 3, Nemotron 3.5 Parse, Docling; Claude: PDF support, vision, Haiku 4.5 price, prompt caching, context windows, Projects RAG, memory, data export; OpenAI: file search, ChatGPT memory; Research: Contextual Retrieval, MemGPT, Lost in the Middle, Context Rot, LongMemEval, Zep, ACE, memory study, Agent Zero Memory, MemFit.

Part 3: Frontend Design and Specification

October 28, 2026

One chat screen serves all four modes. It shows every step live, keeps costs and memory in sight, and builds the safety limits into the controls so a user cannot break anything.

1. Design idea

The app looks calm like a normal chat, but it shows everything the system does: which mode runs, what each agent or step is doing, what it costs, and how full the memory is. Every risky or paid action asks first, so a user cannot break anything by clicking.

- **Show, do not hide:** status lines, thinking, tool calls, agent cards and plan steps appear live.
- **Safe by design:** limits are built into the controls (steppers stop at 2 and 5, locked safety rules), and a blocked action always says why.
- **Costs in sight:** an estimate next to Send, a budget per question, and the reading cost of every file.
- **Plain choices:** Setup shows the real choices directly, each stays visibly selected, and a sentence on top says what will happen.
- **One layout for all modes:** only the Setup dialog and the answer blocks change.

2. Visual style

Black, white and green on a warm ivory ground. Green marks the main action and success; amber and rust are used only for warnings.

Token	Value	Used for
Ground	#FAF9F5 (ivory)	Page background; side menu #F2F0E8
Surface	#FFFFFF	Cards, composer, dialogs
Ink	#141413	Text, Stop button, dark pills
Muted ink	#5E5D59	Hints and meta lines (passes 4.5:1 contrast)
Green	#1F6B4A, hover #13573A	Send, primary buttons, done states, meter
Light green	#E3EFE7 and #F4F9F6	Selected rows, success cards, citations
Amber	#FCEFD6 with #7A4B00 text	Notice, retry, over-limit labels
Rust / dark red	#B4532A / #8E2F20	Alarm meter at 85% / 95%
Lines	#E2DFD4	Borders and dividers
Type	Source Serif 4 (titles), Inter (UI)	Greeting 34 px, dialog titles 21 px, body 14 px
Shape	Radius 10 to 18 px, 40 px minimum targets	Buttons, inputs, cards; no gradients or emoji

3. Screen layout

One screen holds everything; dialogs open on top for models, files, cost, context, export, import and settings.

<table>
<tr><td rowspan="5">Sidebar
New chat
Search chats
Recent chats
Settings, memory, data (one button)
Month spend</td><td>Top bar
Mode switch: Auto, Simple, Thinker, Planner
One Setup button (short summary), Memory meter</td></tr>
<tr><td>Setup dialog (opens from the Setup button)
Direct choices for the mode, plus one sentence: what will happen</td></tr>
<tr><td>Conversation
Empty state: greeting, 4 mode cards, examples
User bubbles with file chips; Auto routing notes
Answers: status line, thinking section, tool chips, citations
Thinker: one progress line, Show agents opens the cards
Planner: plan card, step status, Thinker step notes, approval
Meta line: model, calls, cost; Copy, Good, Bad, Think deeper</td></tr>
<tr><td>Notice or alarm banner
70%: notice; 85%: Summarize, New chat, Download, Later</td></tr>
<tr><td>Composer
File chips (Reading, Ready, open the Document Card)
Message box: Enter sends, Shift+Enter new line
Attach, one cost link (About $X, budget $Y), Send or Stop</td></tr>
</table>

Figure 10. Screen layout. The side menu stacks on top on a phone; dialogs and composer rows wrap.

4. Components

Component	What the user sees and can do
Mode switch	Auto, Simple, Thinker, Planner. Locked while an answer runs, with a dark pill saying so
Model button and picker	Search, provider filter (All, OpenRouter, NVIDIA), must-have filter (Tools, Vision, Thinking). Each row: name, company, provider, badges, cost level, speed. Rows that do not fit the task are grey with the reason
Setup button and dialog	Button shows a short summary. Dialog: a green sentence saying what will happen, then the choices of the mode as cards and switches that stay highlighted (section 5)
Context meter	Percent and a bar: green, amber at 70%, rust at 85%, red at 95%. Click: breakdown by part and actions
User message	Bubble on the right with file chips
Answer header	Model or team name, mode tag, live status line with a dot
Thinking section	Collapsed line "Thought for 4 seconds"; opens to short reasoning notes
Tool chip	"Web search: running" then "3 sources used"; citations [1] [2] under the answer
Team progress line	"3 x Sonnet, together, 2 rounds", round and how many agents are ready; Show agents opens one card per agent
Composer card	Green card: composer model and status; explains how the answer was merged
Plan card	Numbered steps: task, mode, done-test, status pill; retry note in amber; Approve, Edit, Cancel
Output card	File name, Checked pill, pages, sources, Download
Meta line	Model or team, number of calls, cost; Copy, Good, Bad; Think deeper after an Auto answer
Composer	File chips, message box, Attach, one cost link that opens the budget choice ($0.05, $0.25, $1), Send or Stop
Banners and toast	Notice at 70%, alarm at 85% with 4 buttons; short black toast after each action

5. Every mode, every control

Mode	Choices in Setup	What the answer shows
Auto	Quality: Fast, Balanced, Best; the routing rule in one line	A note: "Auto chose Thinker: it compares options"; Think deeper button under Simple answers
Simple	Model; thinking Off, Low, Medium, High (off and grey if the model cannot think); style Precise, Balanced, Creative (temperature 0.2, 0.7, 1.0); tools Web search, Code sandbox, File reader (grey if the model has no tools)	Status line, thinking section, tool chip, streamed answer, citations, meta line
Thinker	Which models: Same model (copies, 2 to 5) or Different models (2 to 5 picked); How they think: On their own (default) or Together (interleaved: draft alone, then read each other and revise); rounds 1 to 3; composer	One progress line, agent cards on demand, merged answer
Planner	Plan writer; steps run in Auto, Simple or Thinker; and when a step uses Thinker, the full Thinker options: same model or different models, agents, on their own or together, rounds, composer. Locked: approval, 2 retries, 2 re-plans, 20 steps	Plan card marking each step easy, hard or tool step, with the cost (team included); live steps, a note per round, retry note (team step: composer first), output file

6. Life of one message

Every send passes the same checks; the user can stop at any moment and keeps what was done.

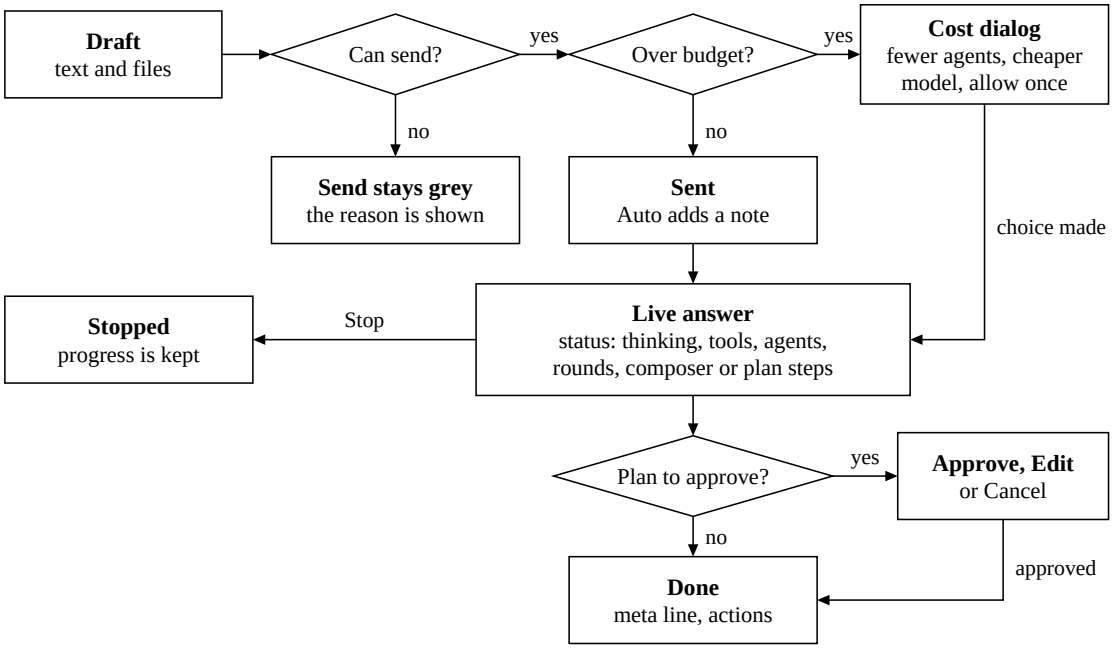


Figure 11. Life of one message: checks before sending, a live answer, approval in Planner, and Stop at any time.

7. Files, memory and context in the interface

Feature	Interaction
Add a file	Attach opens a dialog. After choosing a file, every page is checked for free and shown by type: normal, scanned, chart, each with its reader and cost. Buttons: Read file (cost), Text only (free), Back
Read before	Same file and reader: a green card says the saved reading is reused, cost $0
File chip	Reading... then Ready; click opens the Document Card (pages, readers, cost, summary, outline, tables)
Context details	Parts of the window with bars, the 70, 85, 95% levels and the price of the next message
Alarm	Summarize now, New chat with summary, Download chat, Later (asks again at 90%)
Summarizing	Dialog with 4 checked steps; then a note: older messages summarized, nothing deleted
Download chat	Lists the parts of the .zip; switches: include original files, lock with password
Import chat	4 steps tick off (check, treat as data, reuse readings, load), then a report and Open the chat
Memory	List of remembered facts with date and source; Delete asks Keep or Delete first

8. Settings

Tab	Options
Files (OCR)	Auto, Text layer only, Local open-source, OCR API, Vision model; ask before paying over $0.10, $0.50 or $2; monthly budget $5, $20 or $50; chart descriptions; keep page images
Memory	Long-term memory on or off; auto-summary at 95% or ask; list of facts
Data and safety	Download chat, Import chat, temporary chat, Reset to safe defaults. Safety limits are fixed and shown in Setup

Changes save at once and the footer says so; there is no hidden Save button to forget.

9. Guardrails: the user cannot break it

ID	If the user	The interface
F1	sends an empty message	keeps Send grey
F2	picks fewer than 2 models for Thinker or a Planner team	blocks Send and shows "Pick at least 2 models"
F3	tries more than 5 agents or 3 rounds	stops the stepper and shows the limit in a toast
F4	turns tools on	greys out models without tools, with the reason
F5	attaches a file with charts	greys out models that cannot see images
F6	picks a model that cannot think	sets thinking to Off and explains
F7	sends a question over budget	opens the cost dialog before anything is spent
F8	attaches a file over the ask-me limit	marks the estimate in amber and offers Text only
F9	changes mode while it runs	locks the mode switch until the answer ends or Stop
F10	starts a plan	runs nothing until Approve
F11	deletes a memory	asks Keep or Delete first
F12	changes many settings	offers Reset to safe defaults; safety rules stay locked
F13	mixes strong and weak models in a team	shows a warning and suggests models of similar strength

10. States, access and screen sizes

- **States for every block:** empty, waiting, live, done, stopped, error (with Retry), and offline (message kept as a draft).
- **Keyboard and screen readers:** real buttons, inputs and labels; dialogs named; icon buttons have labels; Enter sends.
- **Contrast:** all text at least 4.5:1; states differ by text and shape, not only by color.
- **Phone:** side menu stacks on top, choice cards and composer rows wrap, tables scroll inside a box.

11. Frontend structure (high level)

Part	Choice	Job
App	Next.js (React), one page for the chat	Layout, routing between chats
State	One store per chat: mode, options, messages, files, context, dialogs	Every component reads the same truth
Live updates	The server sends small events: status, text, tool, agent, step, cost, done	Status lines and cards change live
Components	One component per block in section 4	Same parts reused in every mode
Settings	Saved per user; safe defaults built in	Same behavior on every device
Tests	Each guardrail F1 to F13 and each mode flow as a click test	Nothing breaks after a change

The clickable prototype of this interface is in the Design canvas "Multi-Mode Assistant Frontend". Its model list, answers and prices are sample data.

Multi-Mode AI Assistant - Specification v2 Addendum

Multi-Mode AI Assistant: model, prompt and cost addendum (v2)

Updated November 11, 2026 (Asia/Seoul). This addendum extends the original 15-page Multi-Mode_AI_Assistant_Complete_Spec (1).pdf. It changes model visibility, displayed reasoning, prompt transparency and cost information. Fixed safety limits, approved spending caps and the original mode behavior remain in force.

A1. Complete OpenRouter model catalog

• Show every model returned by the current OpenRouter model catalog, with search and provider filters. The initial list must not be limited to featured models. A refresh may update the catalog.

• Show model identity, provider, company, context limit, tool/image/thinking support, input/output prices, availability and any reason a model cannot be selected.

• Keep models with unknown prices or unavailable providers visible, but block selection for paid runs until a valid price and provider are available. Do not treat an unknown price as zero.

Acceptance: the count and IDs returned by /api/models match the loaded OpenRouter catalog plus configured NVIDIA entries; searching and provider filtering work; an unavailable model has an explicit reason and cannot bypass the budget guard.

A2. Thinking in every mode

• The app may request bounded reasoning from models that support it in Simple, Thinker agents, the Thinker composer and Planner steps/teams. A model without the feature must still answer normally.

• Show each call's role/stage and model. Display reasoning notes only when the provider actually supplies them. When a provider supplies no reasoning text, say that notes were unavailable; never invent or reconstruct private reasoning.

• A reasoning request consumes tokens inside the same approved question budget. Reserve its maximum before dispatch and allow enough separate answer tokens to produce a visible response. Retain existing retry, fallback and Stop behavior.

Acceptance: a capable Thinker agent and composer request bounded reasoning and keep a nonempty answer allowance; an incapable model does not request the feature. Simple, Thinker and Planner display the provider's returned notes under the correct model/agent when available. Missing notes show an unavailable state. No run exceeds its approved cap because of reasoning.

A3. Reviewable prompts for each agent

• Offer a read-only view of the shipped prompt templates: shared answer rules; Simple retry; Thinker agent, Solver, Critic, Fact-checker, Simplifier, Devil's advocate, independent agent, shared board, judge and composer; Planner plan writer, step and output formats; router and Checker; and supporting file/memory templates.

• Label whether each template is a system message, system fragment, user follow-up or response template. Explain that an effective call composes the shared rules, role template and runtime context. Show $placeholders as literal placeholders.

• The view must not return rendered user messages, uploaded documents, memories, tool outputs, API keys or another chat's content. A static template can be read even while the provider is offline.

Acceptance: GET /api/prompts returns the versioned static catalog with distinct IDs, titles, role labels and template text. It never inserts private runtime values. The UI lets the user read the shared rules and every team role's template.

A4. Actual charge after each model call

• After a call finishes, show its model, role or stage, input tokens, output tokens, any reported reasoning tokens, charged USD and whether usage was estimated. Include retries, Checker, router, agents, composer and Planner calls in the run total.

• Failed or cancelled calls whose provider charge is unknown must be labelled as estimated. The answer's cost line must be the sum of recorded call charges rather than a projected price.

• Usage events contain metadata only, never prompt bodies or credentials.

Acceptance: each completed call emits a call_usage event and the final answer exposes total calls and USD. An estimated charge is visibly distinct from a provider-reported charge. Running two agents concurrently preserves each agent's role label without mixing their costs.

A5. Tentative task-size prices for every model

• Show three approximate **single-call** price bands beside each priced model: small 1,000 to 5,000 total tokens, medium 5,000 to 50,000, high 50,000 to 500,000. Use current input/output prices and a documented 70% input, 30% output assumption; include per-request fees when published.

• Show a range of USD for the endpoints of each band. If the model's context cannot hold the whole band, identify the limit and do not present the full band as feasible. If a model cannot fit the minimum or its price is unknown, show unavailable instead of a numeric price.

• Label these bands **tentative**. They exclude cache discounts, image/tool usage, reasoning extras, retries, Checker and other agents. The question-specific estimate and approved cap still govern an actual run.

Acceptance: each priced model API row includes a small/medium/high estimate with token range, price range and feasibility. Test the formula against known model prices, an unknown-price model and a model with a short context window. The interface distinguishes tentative bands from actual post-call charges.

A6. Verification and portability

• Offline tests cover catalog completeness, pricing math, role-labelled reasoning/usage events, prompt catalog privacy and Thinker reasoning capability handling.

• The updated README explains the new controls and their limits. A versioned PDF combines the original specification and this addendum; the original PDF is preserved unchanged.

Acceptance: both the versioned PDF and this Markdown addendum are available in the repository, and their requirements match the implemented API/UI contracts.